NetRunner

DSL para simular secuencias de ciberataques, escaneo de red y evasión de sistemas de defensa

Fase 2: Estructura Sintáctica · Fase 3: Implementación y Parser

Miguel Ángel Vargas Pelaez
Willinton Vergara Cataño
Juan Pablo López Gómez

Profesora Ana María Tamayo Ocampo



Teoría de Lenguajes Formales
Ingeniería de Sistemas y Computación
Facultad de Ingeniería
Universidad del Quindío
2026-2

Tabla de Contenido

1. Introducción **3**

2. Fase 2: Estructura Sintáctica **4**

2.1 Del nivel léxico al nivel sintáctico . . . 4

2.2 Gramática inicial G0 (primer borrador) . . . 4

2.3 Demostración de que G0 es ambigua . . . 5

2.4 Paso 1: eliminación de la ambigüedad (G0 → G1) . . . 7

2.5 Paso 2: eliminación de la recursividad por la izquierda . . . 8

2.6 Paso 3: factorización por la izquierda . . . 9

2.7 Paso 4: verificación de gramática bien formada . . . 9

2.8 Gramática final G2 . . . 10

2.9 Demostración de que G2 no es ambigua (condición LL(1)) . . . 12

2.10 Derivación más a la izquierda y árbol de derivación . . . 14

3. Fase 3: Implementación y Parser **17**

3.1 Arquitectura . . . 17

3.2 Analizador léxico: de los AFD al código . . . 17

3.3 Analizador sintáctico descendente recursivo LL(1) . . . 18

3.4 Construcción y salida del árbol de derivación . . . 19

3.5 Manejo de errores . . . 20

3.6 Plan de pruebas . . . 21

3.7 Verificación frente a la rúbrica (Arquitectura de software) . . . 21

3.8 Trazabilidad entre fases . . . 22

4. Guía de sustentación e ingeniería inversa **23**

4.1 Rastreo manual de tokens en el AFD . . . 23

4.2 Modificación de la gramática en vivo: ejemplo resuelto . . . 23

4.3 Preguntas probables y respuesta corta . . . 24

5. Ajustes de consistencia recomendados para la Fase 1 **24**

6. Referencias **24**

1. Introducción

Este documento continúa la Fase 1 (Formalismo Léxico) del proyecto NetRunner y desarrolla las dos fases restantes. La **Fase 2** (núcleo temático 4) define la Gramática Independiente del Contexto (GIC) del lenguaje en notación BNF y demuestra, paso a paso, la eliminación de la ambigüedad, de la recursividad por la izquierda y la factorización por la izquierda, hasta obtener una gramática LL(1), que por construcción no es ambigua. La **Fase 3** describe el analizador sintáctico implementado en Python: cómo los AFD de la Fase 1 se convierten en el analizador léxico, cómo cada regla de la GIC se convierte en una función del parser, cómo se construye el árbol de derivación y cómo se reportan los errores.

La idea que conecta las tres fases es la siguiente: el analizador léxico agrupa los caracteres del alfabeto $\Sigma_{\text{NetRunner}}$ en tokens; esos tokens son los **símbolos terminales** de la gramática. Es decir, el alfabeto de la GIC no es el de caracteres de la Fase 1, sino el conjunto de categorías léxicas que esa fase reconoce. Así se evita mezclar el alfabeto con la gramática.

2. Fase 2: Estructura Sintáctica

2.1 Del nivel léxico al nivel sintáctico

Cada token de la Fase 1 pasa a ser un terminal de la GIC. Los comentarios (q16) y los espacios en blanco (q17) son reconocidos por el analizador léxico pero se descartan, por lo que no aparecen en la gramática.

Tabla 1. Correspondencia entre tokens de la Fase 1 y terminales de la GIC

Token (Fase 1)	Estado de aceptación	Terminal en la GIC
TK_CMD (23 palabras)	hojas del trie	`TARGET, NODE, IF, THEN, ELSE, SCAN, PORTS, CONNECT, PORT, BYPASS, ICE_LEVEL, DISABLE, FIREWALL, ACTIVE, INACTIVE, INJECT, PAYLOAD, EXECUTE, SCRIPT, EXFIL, DATA, CLEAR, LOGS (un terminal por palabra)`
TK_NUM	q1, q2	`NUM`
HEX_ADDR	q4	`HEX_ADDR`
STRING	q6	`STRING`
TK_MAYOR / TK_MENOR / TK_IGUAL	q7 / (nuevo) / q9	`> < ==`
TK_OP	q11	`->`
TK_IZQ / TK_DER	q12 / q13	`{ }`
TK_DELIM	q14	`;`
COMENTARIO / ESPACIO	q16 / q17	`(se descartan)`

Cada palabra reservada se trata como un terminal propio (y no como un único terminal TK_CMD) porque la gramática necesita distinguir, por ejemplo, IF de SCAN para decidir qué regla aplicar.

¿Por qué se necesita una GIC y no basta una expresión regular? Los bloques de NetRunner pueden anidarse (un IF dentro de otro IF), y cada { debe cerrarse con su }. Ese patrón tiene la misma forma que el lenguaje $\{ a^n b^n \mid n \geq 1 \}$ visto en clase: para reconocerlo hay que recordar cuántas llaves se abrieron, y un AFD solo tiene un número finito de estados. Una GIC sí lo describe gracias a la recursividad (*bloque → ... instrucción → condicional → ... bloque*).

2.2 Gramática inicial G0 (primer borrador)

A partir del enunciado y de los scripts de ejemplo se escribió una primera gramática, la más natural posible. Se usa la cuádrupla $G = (Z_T, Z_N, S, P)$ del curso, con ::= ("se define como"), | ("o") y ε para la cadena vacía.

```
G0-1  <programa>    ::= TARGET NODE HEX_ADDR { <lista> }
G0-2  <lista>       ::= <lista> <instruccion> | <instruccion>
G0-3  <instruccion> ::= <secuencia> ; | <condicional>
G0-4  <secuencia>   ::= <secuencia> -> <secuencia> | <accion>
G0-5  <accion>      ::= SCAN PORTS | CONNECT PORT NUM | BYPASS FIREWALL
                      | DISABLE FIREWALL | INJECT PAYLOAD STRING
                      | EXECUTE SCRIPT STRING | EXFIL DATA | CLEAR LOGS
G0-6  <condicional> ::= IF <condicion> THEN <cuerpo>
                      | IF <condicion> THEN <cuerpo> ELSE <cuerpo>
```

```
G0-7  <cuerpo>      ::= { <lista> } | <instruccion>
G0-8  <condicion>   ::= ICE_LEVEL > NUM | ICE_LEVEL < NUM | ICE_LEVEL == NUM
                      | FIREWALL == ACTIVE | FIREWALL == INACTIVE
```

Tabla 2. Diagnóstico de la gramática G0

Regla	Problema detectado	Consecuencia
G0-4	Ambigüedad por asociatividad: <secuencia> aparece a ambos lados del operador ->	Una cadena con tres acciones tiene dos árboles distintos (sección 2.3)
G0-4	Recursividad directa por la izquierda (<secuencia> ::= <secuencia> ...)	Un parser descendente entra en un ciclo infinito
G0-2	Recursividad directa por la izquierda (<lista> ::= <lista> ...)	Igual que el anterior
G0-6 / G0-7	"ELSE colgante": el cuerpo puede ser una instrucción sin llaves	Un ELSE puede asociarse a dos IF distintos (ambigüedad)
G0-6	Prefijo común IF <condicion> THEN <cuerpo> en las dos alternativas	Con un token de anticipación no se sabe qué alternativa elegir
G0-8	Prefijos comunes ICE_LEVEL y FIREWALL ==	Mismo problema de decisión

2.3 Demostración de que G0 es ambigua

Una gramática es ambigua si existe al menos una sentencia con dos o más árboles de derivación distintos (equivalentemente, dos derivaciones más a la izquierda distintas). Basta entonces con exhibir una sentencia.

a) Ambigüedad del operador de secuencia

Sentencia w_1 = SCAN PORTS -> CONNECT PORT NUM -> EXFIL DATA. Se abrevia <S> = <secuencia> y <A> = <accion>. Dos derivaciones más a la izquierda distintas:

```
Derivación 1:  <S> ⇒ <S> -> <S>          (G0-4)
               ⇒ <S> -> <S> -> <S>      (G0-4 sobre el primer <S>)
               ⇒ <A> -> <S> -> <S> ⇒* SCAN PORTS -> CONNECT PORT NUM -> EXFIL DATA

Derivación 2:  <S> ⇒ <S> -> <S>          (G0-4)
               ⇒ <A> -> <S>             (G0-4 alternativa <accion>)
               ⇒ SCAN PORTS -> <S>
               ⇒ SCAN PORTS -> <S> -> <S> ⇒* SCAN PORTS -> CONNECT PORT NUM -> EXFIL DATA
```

La primera agrupa (SCAN -> CONNECT) -> EXFIL y la segunda SCAN -> (CONNECT -> EXFIL). Como las derivaciones más a la izquierda difieren en el segundo paso, los árboles son distintos (Figuras 1 y 2).

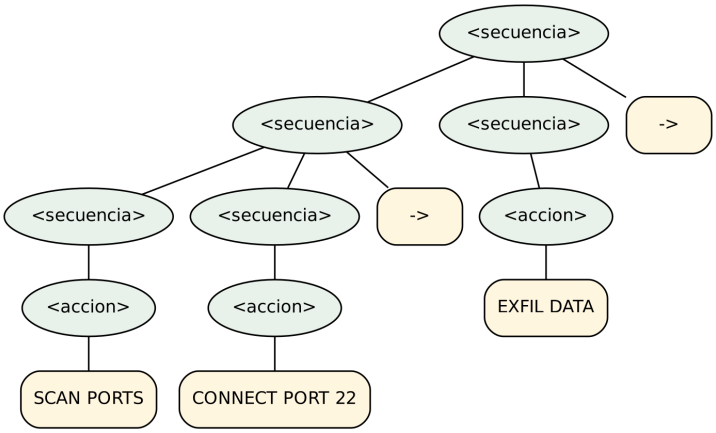


Figura 1. G0, árbol 1 de w_1: agrupación por la izquierda

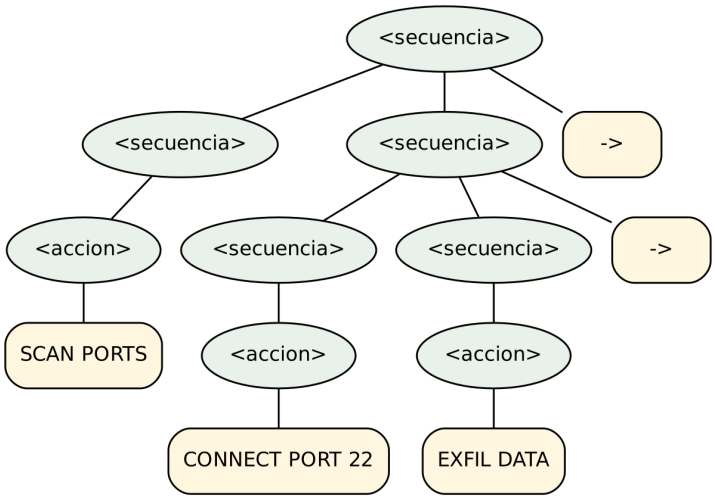


Figura 2. G0, árbol 2 de w_1: agrupación por la derecha

b) Ambigüedad del ELSE colgante

Sentencia w_2 = IF ICE_LEVEL > NUM THEN IF FIREWALL == ACTIVE THEN BYPASS FIREWALL ; ELSE CLEAR LOGS ;. Como el cuerpo de un IF puede ser una instrucción sin llaves (G0-7), el ELSE puede pertenecer al IF interno (Figura 3) o al externo (Figura 4). Semánticamente es grave: en un caso CLEAR LOGS se ejecuta cuando el cortafuegos está inactivo y en el otro cuando el nivel ICE es bajo.

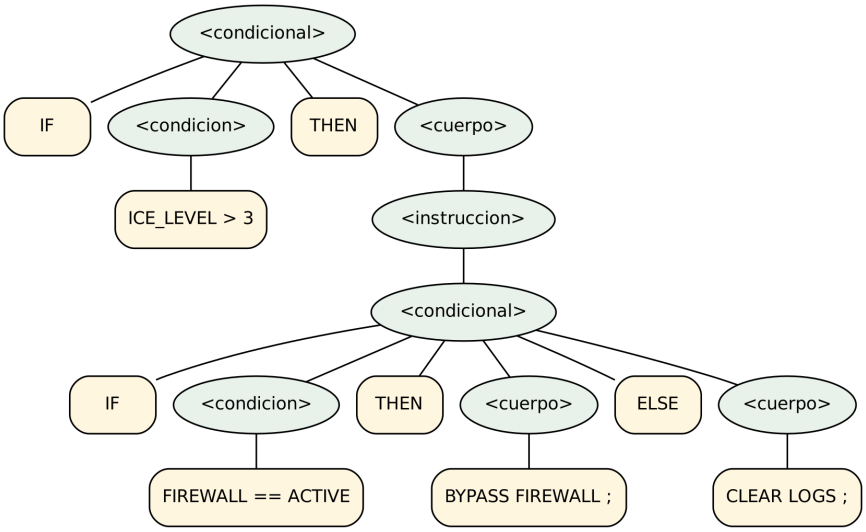


Figura 3. G0, árbol 1 de w_2: el ELSE pertenece al IF interno

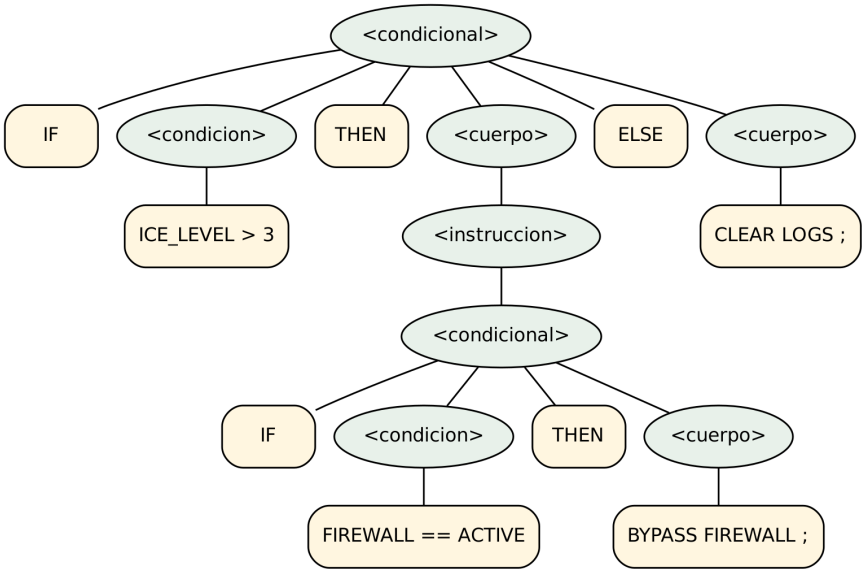


Figura 4. G0, árbol 2 de w_2: el ELSE pertenece al IF externo

2.4 Paso 1: eliminación de la ambigüedad (G0 → G1)

- **Secuencia.** Se reemplaza G0-4 por `<secuencia> ::= <secuencia> -> <accion> | <accion>`. Ahora el lado derecho del operador solo puede ser una acción, lo que fija la asociatividad por la izquierda (las acciones se ejecutan en el orden en que se escriben). Prueba de unicidad por inducción sobre n, el número de acciones: si n = 1 solo aplica la regla <accion>; si n > 1, la última acción solo puede provenir de la regla aplicada en la raíz, y el resto (n − 1 acciones) tiene árbol único por hipótesis de inducción.
- **ELSE colgante.** Se elimina la alternativa <instruccion> de G0-7: el cuerpo de THEN y de ELSE siempre es un bloque con llaves. Las dos lecturas de w_2 pasan a ser cadenas *distintas* (el } de cierre indica a qué IF pertenece el ELSE), por lo que ya no existe una misma sentencia con dos árboles.

- **Regla de redenominación.** Tras el cambio anterior queda `<cuerpo> ::= <bloque>`, una regla A ::= B. Con el algoritmo de eliminación de reglas de redenominación (Gramáticas Bien Formadas) se sustituye <cuerpo> por <bloque> en todas sus apariciones, y se define `<bloque> ::= { <lista> }`, que también reemplaza las llaves de G0-1.

```
G1:  <programa>    ::= TARGET NODE HEX_ADDR <bloque>
     <bloque>      ::= { <lista> }
     <lista>       ::= <lista> <instruccion> | <instruccion>          ← recursiva izq.
     <instruccion> ::= <secuencia> ; | <condicional>
     <secuencia>   ::= <secuencia> -> <accion> | <accion>            ← recursiva izq.
     <accion>      ::= (igual que G0-5)
     <condicional> ::= IF <condicion> THEN <bloque>
                     | IF <condicion> THEN <bloque> ELSE <bloque>     ← prefijo común
     <condicion>   ::= (igual que G0-8)                               ← prefijos comunes
```

2.5 Paso 2: eliminación de la recursividad por la izquierda

Una regla es recursiva por la izquierda si $A \Rightarrow^{+} A\alpha$, es decir, si A puede derivar una forma sentencial que empieza por el mismo A. En un parser descendente recursivo la función de A se llamaría a sí misma sin consumir ningún token, generando una recursión infinita. Por eso debe eliminarse.

Teorema (transformación estándar)

Sea A ::= Aα | β, donde β no empieza por A. Esta regla es equivalente a A ::= βA′, A′ ::= αA′ | ε, con A′ un no terminal nuevo.

Demostración. (i) En la gramática original, toda derivación desde A debe terminar aplicando A ::= β exactamente una vez (es la única regla que elimina A); antes de ella se aplicó A ::= Aα un número k ≥ 0 de veces: $A \Rightarrow A\alpha \Rightarrow A\alpha\alpha \Rightarrow \ldots \Rightarrow A\alpha^{k} \Rightarrow \beta\alpha^{k}$. Luego L(A) = βα*. (ii) En la nueva gramática, por inducción sobre k, $A' \Rightarrow^{*} \alpha^{k}$ (caso base: A′ ⇒ ε; paso: A′ ⇒ αA′ ⇒* $\alpha\alpha^{k}$). Entonces $A \Rightarrow \beta A' \Rightarrow^{*} \beta\alpha^{k}$, de modo que L(A) = βα* también. Ambos lenguajes son iguales. ■

Aplicación a <lista>

Con α = <instruccion> y β = <instruccion> (se renombra <lista> como <lista_instr>):

```
<lista_instr>  ::= <instruccion> <lista_instr'>
<lista_instr'> ::= <instruccion> <lista_instr'> | ε
L(<lista_instr>) = <instruccion>·<instruccion>* = <instruccion>+   (al menos una instrucción)
```

Aplicación a <secuencia>

Con α = -> <accion> y β = <accion>:

```
<secuencia>  ::= <accion> <secuencia'>
<secuencia'> ::= -> <accion> <secuencia'> | ε
L(<secuencia>) = <accion> ( -> <accion> )*
```

Recursividad indirecta. Se descarta construyendo el grafo "puede empezar por": <lista_instr> y <lista_instr′> → <instruccion>; <instruccion> → <secuencia>, <condicional>; <secuencia> → <accion>. Los demás no terminales empiezan por un terminal (TARGET, {, IF, ->, ELSE, ICE_LEVEL, FIREWALL…). El grafo no tiene ciclos, por lo tanto ningún no terminal puede derivarse a sí mismo en la posición más a la izquierda.

2.6 Paso 3: factorización por la izquierda

Teorema. A ::= $\alpha\beta_1$ | $\alpha\beta_2$ es equivalente a A ::= αA′, A′ ::= β_1 | β_2. **Demostración:** la concatenación distribuye sobre la unión de lenguajes: $\alpha(\beta_1 \cup \beta_2) = \alpha\beta_1 \cup \alpha\beta_2$. ■ Con la gramática factorizada el parser consume primero α y solo después decide entre β_1 y β_2.

Aplicación a <condicional>

α = IF <condicion> THEN <bloque>, $\beta_1 = \varepsilon$, β_2 = ELSE <bloque>:

```
<condicional> ::= IF <condicion> THEN <bloque> <resto_if>
<resto_if>    ::= ELSE <bloque> | ε
```

Aplicación a <condicion>

Primero se factoriza el prefijo ICE_LEVEL; las tres alternativas resultantes comparten el sufijo NUM, y por la misma propiedad distributiva (> NUM | < NUM | == NUM) = (> | < | ==) NUM. Después se factoriza FIREWALL ==:

```
<condicion> ::= ICE_LEVEL <op_rel> NUM | FIREWALL == <estado>
<op_rel>    ::= > | < | ==
<estado>    ::= ACTIVE | INACTIVE
```

Las alternativas de <accion> no requieren factorización: cada una empieza por una palabra reservada distinta. PORT y PORTS tampoco generan conflicto sintáctico: son terminales distintos que el analizador léxico separa por la regla del lexema más largo (el trie acepta en PORT pero continúa si sigue una S).

2.7 Paso 4: verificación de gramática bien formada

Se aplican los criterios de la clase de Gramáticas Bien Formadas a la gramática resultante:

Tabla 3. Criterios de gramática bien formada

Criterio	Verificación	Resultado
Reglas innecesarias (A ::= A)	Ninguna regla tiene como lado derecho únicamente su propio lado izquierdo	No hay
Símbolos inaccesibles	Algoritmo de marcado desde el axioma (Tabla 4)	Todos accesibles
Símbolos no generativos / reglas superfluas	Cada no terminal deriva una cadena de terminales (Tabla 5)	No hay
Reglas no generativas (A ::= ε)	Solo en <lista_instr'>, <secuencia'> y <resto_if>; el axioma no deriva ε	Se conservan (ver nota)
Reglas de redenominación (A ::= B)	Solo <instruccion> ::= <condicional>	Se conserva (ver nota)

Tabla 4. Marcado de símbolos accesibles

Iteración	Símbolos marcados
0	<programa> (axioma)
1	TARGET, NODE, HEX_ADDR, <bloque>
2	{, <lista_instr>, }
3	<instruccion>, <lista_instr'>
4	<secuencia>, ;, <condicional>
5	<accion>, <secuencia'>, IF, <condicion>, THEN, <resto_if>
6	SCAN, PORTS, CONNECT, PORT, NUM, BYPASS, FIREWALL, DISABLE, INJECT, PAYLOAD, STRING, EXECUTE, SCRIPT, EXFIL, DATA, CLEAR, LOGS, ->, ELSE, ICE_LEVEL, <op_rel>, ==, <estado>
7	>, <, ACTIVE, INACTIVE
8	Ningún símbolo nuevo → fin del algoritmo

Tabla 5. Marcado de símbolos generativos (de abajo hacia arriba)

Iteración	No terminal generativo	Cadena terminal que lo demuestra
1	<accion>, <op_rel>, <estado>, <secuencia'>, <lista_instr'>, <resto_if>	`SCAN PORTS · > · ACTIVE · ε`
2	<secuencia>, <condicion>	`SCAN PORTS · FIREWALL == ACTIVE`
3	<instruccion>	`SCAN PORTS ;`
4	<lista_instr>	`SCAN PORTS ;`
5	<bloque>	`{ SCAN PORTS ; }`
6	<condicional>, <programa>	`IF FIREWALL == ACTIVE THEN { SCAN PORTS ; } · TARGET NODE HEX_ADDR { SCAN PORTS ; }`

Nota de diseño. Las reglas ε se mantienen a propósito: si se eliminaran con el algoritmo de reglas no generativas, por ejemplo <resto_if>, se volvería a obtener <condicional> ::= IF … <bloque> ELSE <bloque> | IF … <bloque>, es decir, se reintroduciría el prefijo común que se acaba de factorizar. En una gramática LL(1) las reglas ε son las que permiten "decidir no seguir" mirando el conjunto SIGUIENTE. La regla <instruccion> ::= <condicional> también se conserva: no es cíclica, no genera ambigüedad y da un nodo con nombre propio en el árbol; si se eliminara, quedaría <instruccion> ::= <secuencia> ; | IF <condicion> THEN <bloque> <resto_if>, una gramática equivalente y también LL(1).

2.8 Gramática final G2

G2 = (Z_T, Z_N, S, P), donde:

- **Z_T** (33 terminales) = { TARGET, NODE, HEX_ADDR, {, }, ;, ->, IF, THEN, ELSE, ICE_LEVEL, FIREWALL, ACTIVE, INACTIVE, >, <, ==, NUM, STRING, SCAN, PORTS, CONNECT, PORT, BYPASS, DISABLE, INJECT, PAYLOAD, EXECUTE, SCRIPT, EXFIL, DATA, CLEAR, LOGS }
- **Z_N** (13 no terminales) = { <programa>, <bloque>, <lista_instr>, <lista_instr'>, <instruccion>, <secuencia>, <secuencia'>, <accion>, <condicional>, <resto_if>, <condicion>, <op_rel>, <estado> }

- **S** = <programa>
- **P** = las 28 reglas siguientes (R1 a R28):

Tabla 6. Reglas de producción de la gramática final G2

Regla	Producción (BNF)
R1	<programa> ::= TARGET NODE HEX_ADDR <bloque>
R2	<bloque> ::= { <lista_instr> }
R3	<lista_instr> ::= <instruccion> <lista_instr'>
R4	<lista_instr'> ::= <instruccion> <lista_instr'>
R5	<lista_instr'> ::= ε
R6	<instruccion> ::= <secuencia> ;
R7	<instruccion> ::= <condicional>
R8	<secuencia> ::= <accion> <secuencia'>
R9	<secuencia'> ::= -> <accion> <secuencia'>
R10	<secuencia'> ::= ε
R11	<accion> ::= SCAN PORTS
R12	<accion> ::= CONNECT PORT NUM
R13	<accion> ::= BYPASS FIREWALL
R14	<accion> ::= DISABLE FIREWALL
R15	<accion> ::= INJECT PAYLOAD STRING
R16	<accion> ::= EXECUTE SCRIPT STRING
R17	<accion> ::= EXFIL DATA
R18	<accion> ::= CLEAR LOGS
R19	<condicional> ::= IF <condicion> THEN <bloque> <resto_if>
R20	<resto_if> ::= ELSE <bloque>
R21	<resto_if> ::= ε
R22	<condicion> ::= ICE_LEVEL <op_rel> NUM
R23	<condicion> ::= FIREWALL == <estado>
R24	<op_rel> ::= >
R25	<op_rel> ::= <
R26	<op_rel> ::= ==
R27	<estado> ::= ACTIVE
R28	<estado> ::= INACTIVE

Versión E-BNF equivalente (útil para la implementación)

```
<programa>    ::= 'TARGET' 'NODE' HEX_ADDR <bloque>
<bloque>      ::= '{' <instruccion>+ '}'
<instruccion> ::= <secuencia> ';' | <condicional>
<secuencia>   ::= <accion> ( '->' <accion> )*
<accion>      ::= 'SCAN' 'PORTS' | 'CONNECT' 'PORT' NUM | 'BYPASS' 'FIREWALL'
                | 'DISABLE' 'FIREWALL' | 'INJECT' 'PAYLOAD' STRING
                | 'EXECUTE' 'SCRIPT' STRING | 'EXFIL' 'DATA' | 'CLEAR' 'LOGS'
<condicional> ::= 'IF' <condicion> 'THEN' <bloque> ( 'ELSE' <bloque> )?
<condicion>   ::= 'ICE_LEVEL' ( '>' | '<' | '==' ) NUM
                | 'FIREWALL' '==' ( 'ACTIVE' | 'INACTIVE' )
```

Cada **+** o ***** de la E-BNF corresponde a un no terminal primado de la BNF (<lista_instr′>, <secuencia′>) y se implementa con un ciclo; cada **?** corresponde a una regla con alternativa ε (<resto_if>) y se implementa con un if.

2.9 Demostración de que G2 no es ambigua (condición LL(1))

La ambigüedad es indecidible en general: no existe un algoritmo que la detecte para cualquier gramática. Por eso se usa una **condición suficiente**: toda gramática LL(1) es no ambigua. La razón es que en una gramática LL(1) el siguiente token determina de forma única la regla a aplicar en cada paso de la derivación más a la izquierda; luego cada sentencia tiene una sola derivación más a la izquierda y, por lo tanto, un solo árbol.

Conjuntos PRIMERO y SIGUIENTE

PRIMERO(α) es el conjunto de terminales con que puede empezar α (incluye ε si α ⇒* ε). SIGUIENTE(A) es el conjunto de terminales que pueden aparecer inmediatamente después de A ($ = fin de archivo). Se abrevia **ACC** = { SCAN, CONNECT, BYPASS, DISABLE, INJECT, EXECUTE, EXFIL, CLEAR }.

Tabla 7. Conjuntos PRIMERO y SIGUIENTE de G2

No terminal	PRIMERO	SIGUIENTE
<programa>	{ TARGET }	{ $ }
<bloque>	{ { }	{ ACC, IF, ELSE, }, $ }
<lista_instr>	{ ACC, IF }	{ } }
<lista_instr'>	{ ACC, IF, ε }	{ } }
<instruccion>	{ ACC, IF }	{ ACC, IF, } }
<secuencia>	{ ACC }	{ ; }
<secuencia'>	{ ->, ε }	{ ; }
<accion>	{ ACC }	{ ;, -> }
<condicional>	{ IF }	{ ACC, IF, } }
<resto_if>	{ ELSE, ε }	{ ACC, IF, } }
<condicion>	{ ICE_LEVEL, FIREWALL }	{ THEN }
<op_rel>	{ >, <, == }	{ NUM }
<estado>	{ ACTIVE, INACTIVE }	{ THEN }

Cálculo de algunos SIGUIENTE (los demás se obtienen igual): SIGUIENTE(<programa>) = {$} por ser el axioma. Por R2, } ∈ SIGUIENTE(<lista_instr>). Por R3 y R4, SIGUIENTE(<instruccion>) =

(PRIMERO(<lista_instr′>) − {ε}) ∪ SIGUIENTE(<lista_instr′>) = ACC ∪ {IF} ∪ {}}. Por R6, SIGUIENTE(<secuencia>) = {;}. Por R8 y R9, SIGUIENTE(<accion>) = {->} ∪ SIGUIENTE(<secuencia′>) = {->, ;}. Por R19, SIGUIENTE(<bloque>) incluye {ELSE} ∪ SIGUIENTE(<condicional>), y por R1 incluye $.

Condiciones LL(1)

Para toda regla A ::= α | β se debe cumplir: (1) PRIMERO(α) ∩ PRIMERO(β) = ∅ y (2) si β ⇒* ε, entonces PRIMERO(α) ∩ SIGUIENTE(A) = ∅.

Tabla 8. Verificación de las condiciones LL(1) en G2

No terminal	PRIMERO de cada alternativa	Verificación
<lista_instr'>	R4: { ACC, IF } | R5: { ε }	PRIMERO ∩ SIGUIENTE{ } } = ∅ ✔
<instruccion>	R6: { ACC } | R7: { IF }	Alternativas disjuntas = ∅ ✔
<secuencia'>	R9: { -> } | R10: { ε }	PRIMERO ∩ SIGUIENTE{ ; } = ∅ ✔
<accion>	R11: { SCAN } | R12: { CONNECT } | R13: { BYPASS } | R14: { DISABLE } | R15: { INJECT } | R16: { EXECUTE } | R17: { EXFIL } | R18: { CLEAR }	Alternativas disjuntas = ∅ ✔
<resto_if>	R20: { ELSE } | R21: { ε }	PRIMERO ∩ SIGUIENTE{ ACC, IF, } } = ∅ ✔
<condicion>	R22: { ICE_LEVEL } | R23: { FIREWALL }	Alternativas disjuntas = ∅ ✔
<op_rel>	R24: { > } | R25: { < } | R26: { == }	Alternativas disjuntas = ∅ ✔
<estado>	R27: { ACTIVE } | R28: { INACTIVE }	Alternativas disjuntas = ∅ ✔

Tabla de análisis predictivo M[A, a]

M[A, a] = Rn significa: si el parser está expandiendo A y el siguiente token es a, aplica la regla Rn. Las celdas no listadas están vacías y corresponden a un error sintáctico. Ninguna celda tiene dos reglas, lo cual confirma que G2 es LL(1).

Tabla 9. Tabla de análisis LL(1) de G2 (forma compacta)

A	Token de anticipación a → regla
<programa>	TARGET → R1
<bloque>	{ → R2
<lista_instr>	ACC, IF → R3
<lista_instr'>	ACC, IF → R4 } → R5
<instruccion>	ACC → R6 IF → R7
<secuencia>	ACC → R8
<secuencia'>	-> → R9 ; → R10
<accion>	SCAN → R11 CONNECT → R12 BYPASS → R13 DISABLE → R14 INJECT → R15 EXECUTE → R16 EXFIL → R17 CLEAR → R18
<condicional>	IF → R19
<resto_if>	ELSE → R20 ACC, IF, } → R21
<condicion>	ICE_LEVEL → R22 FIREWALL → R23
<op_rel>	> → R24 < → R25 == → R26
<estado>	ACTIVE → R27 INACTIVE → R28

Revisión de las sentencias problemáticas. En G2, w_1 solo admite la derivación <secuencia> ⇒ <accion> <secuencia′> ⇒ … con un único árbol que crece hacia la derecha; y el ELSE colgante desaparece porque THEN y ELSE exigen <bloque>, y M[<resto_if>, ELSE] = R20 es la única entrada para ELSE.

2.10 Derivación más a la izquierda y árbol de derivación

Se analiza el siguiente programa P_1:

```
TARGET NODE 0xA1 {
    SCAN PORTS -> CONNECT PORT 22;
    IF FIREWALL == ACTIVE THEN {
        BYPASS FIREWALL;
    }
}
```

El analizador léxico lo convierte en la cadena de terminales: TARGET NODE HEX_ADDR { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN { BYPASS FIREWALL ; } }. Para que la tabla sea legible se abrevian los no terminales: P = programa, B = bloque, L = lista_instr, L′ = lista_instr′, I = instruccion, S = secuencia, S′ = secuencia′, A = accion, C = condicional, R = resto_if, K = condicion, E = estado. El prefijo TARGET NODE HEX_ADDR se escribe como T·N·H.

Tabla 10. Derivación más a la izquierda de P_1 (23 pasos)

Paso	Forma sentencial (se expande siempre el no terminal más a la izquierda)	Regla
0	`<P>`	—
1	`T·N·H <B>`	R1
2	`T·N·H { <L> }`	R2
3	`T·N·H { <I> <L'> }`	R3
4	`T·N·H { <S> ; <L'> }`	R6
5	`T·N·H { <A> <S'> ; <L'> }`	R8
6	`T·N·H { SCAN PORTS <S'> ; <L'> }`	R11
7	`T·N·H { SCAN PORTS -> <A> <S'> ; <L'> }`	R9
8	`T·N·H { SCAN PORTS -> CONNECT PORT NUM <S'> ; <L'> }`	R12
9	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; <L'> }`	R10
10	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; <I> <L'> }`	R4
11	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; <C> <L'> }`	R7
12	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF <K> THEN <B> <R> <L'> }`	R19
13	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == <E> THEN <B> <R> <L'> }`	R23
14	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN <B> <R> <L'> }`	R27
15	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN { <L> } <R> <L'> }`	R2
16	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN { <I> <L'> } <R> <L'> }`	R3
17	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN { <S> ; <L'> } <R> <L'> }`	R6
18	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN { <A> <S'> ; <L'> } <R> <L'> }`	R8
19	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN { BYPASS FIREWALL <S'> ; <L'> } <R> <L'> }`	R13
20	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN { BYPASS FIREWALL ; <L'> } <R> <L'> }`	R10
21	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN { BYPASS FIREWALL ; } <R> <L'> }`	R5
22	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN { BYPASS FIREWALL ; } <L'> }`	R21
23	`T·N·H { SCAN PORTS -> CONNECT PORT NUM ; IF FIREWALL == ACTIVE THEN { BYPASS FIREWALL ; } }`	R5

Leyendo las hojas del árbol de izquierda a derecha (su frontera) se obtiene exactamente la cadena de entrada. Los nodos verdes son no terminales (nodos interiores) y los amarillos son terminales (hojas), con su lexema real.

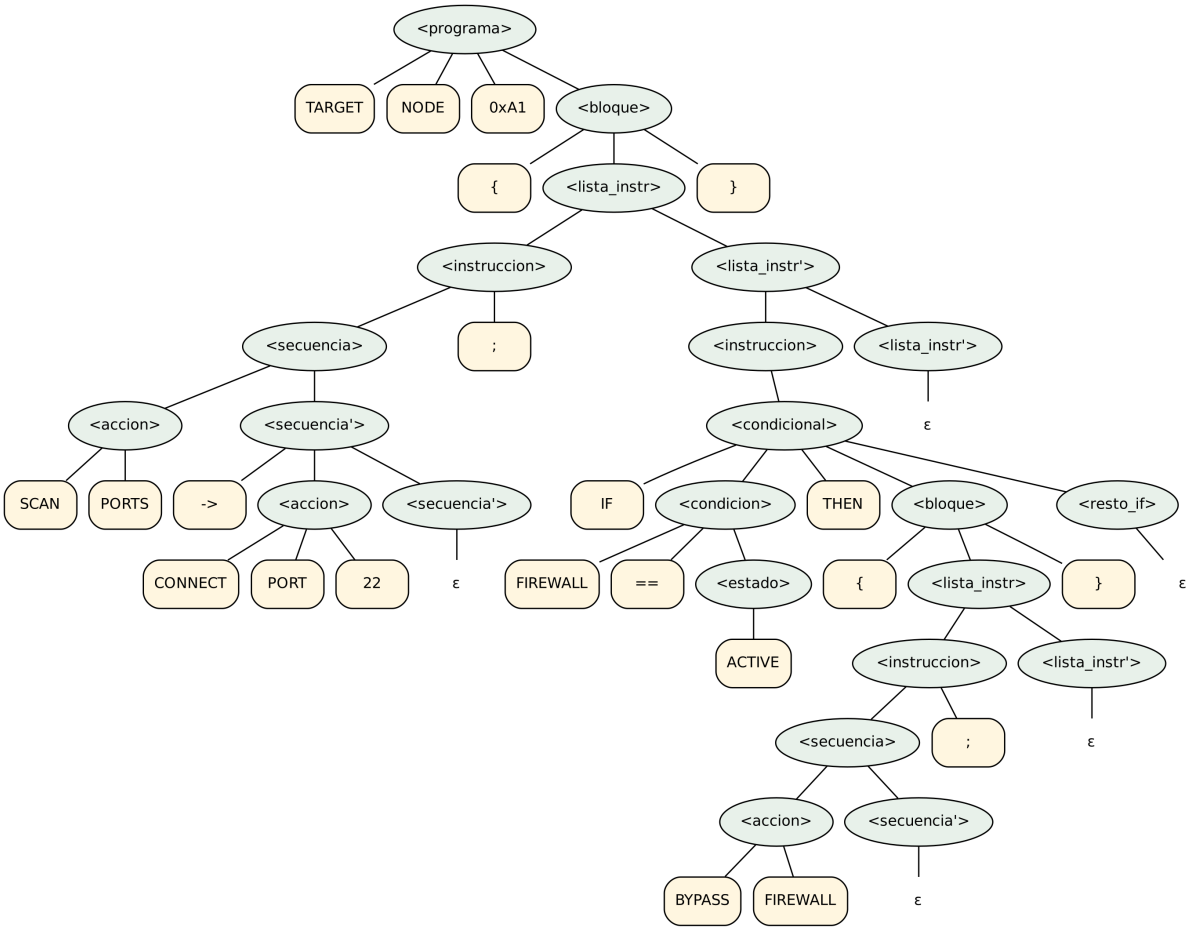


Figura 5. Árbol de derivación sintáctica de P_1 según G2

3. Fase 3: Implementación y Parser

3.1 Arquitectura

El programa recibe la ruta de un archivo de texto con código NetRunner y lo procesa en dos etapas encadenadas. El analizador léxico produce una lista de tokens con su posición; el analizador sintáctico la recorre una sola vez, de izquierda a derecha, y produce el árbol de derivación o el listado de errores.

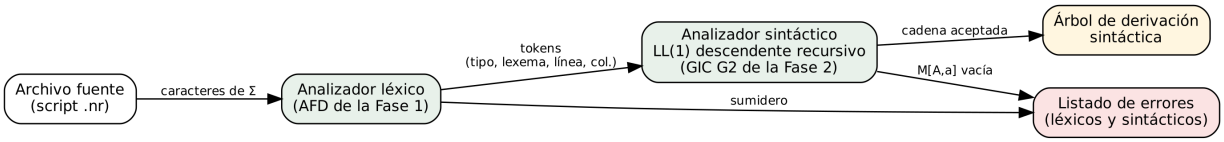


Figura 6. Flujo de procesamiento del analizador NetRunner

Tabla 11. Componentes del sistema y su relación con la teoría

Componente	Responsabilidad	Base teórica
Analizador léxico	Lee caracteres, aplica los AFD y emite tokens (tipo, lexema, línea, columna); descarta comentarios y espacios	Fase 1: Σ, ER y AFD
Analizador sintáctico	Una función por no terminal de G2; elige la regla con el token de anticipación	Fase 2: G2, PRIMERO, SIGUIENTE, tabla M
Árbol de derivación	Nodos interiores = no terminales; hojas = terminales o ε; impresión jerárquica	Clase de árboles de derivación
Reporte de errores	Mensajes con línea, columna, esperado y encontrado; recuperación en modo pánico	Estados de rechazo y celdas vacías de M
Programa principal	Recibe la ruta del archivo por línea de comandos y muestra el resultado	Evita rutas fijas ("hardcodeo")

Los nombres de módulos, clases y funciones de esta sección son de referencia. Antes de la sustentación, ajústenlos a los nombres reales del código (la auditoría con Claude Code los reporta).

3.2 Analizador léxico: de los AFD al código

- **Regla del lexema más largo (maximal munch).** Desde q0 se avanza mientras exista transición; al no haberla, se emite el token del último estado de aceptación visitado. Así PORTS se reconoce como PORTS y no como PORT + S.
- **Palabras reservadas.** El trie de 111 estados (Tabla 1 de la Fase 1) se implementa como diccionario de transiciones o, de forma equivalente, leyendo la secuencia de letras/guion bajo y verificando su pertenencia al conjunto de las 23 palabras. Una palabra que no está (por ejemplo, scan) equivale a caer en el sumidero.
- **Números y direcciones.** El AFD q0–q4 separa NUM (q1, q2) de HEX_ADDR (q4); 0x sin dígitos termina en q3, estado de paso, y es error léxico.
- **Operadores.** - debe ir seguido de > (q10 → q11) y = debe ir seguido de = (q8 → q9); un - o un = solos terminan en estado de paso y producen error léxico.
- **Cadenas y comentarios.** " abre la cadena (q5) y otra " la cierra (q6); fin de línea o de archivo dentro de q5 es "cadena sin cerrar". // lleva a q16 hasta el salto de línea y el

comentario se descarta.

3.3 Analizador sintáctico descendente recursivo LL(1)

Se implementa la técnica de descenso recursivo: **una función por cada no terminal** de G2. Dentro de cada función, la alternativa se elige comparando el token actual con los conjuntos PRIMERO (o SIGUIENTE, para las alternativas ε) de la Tabla 7; los terminales se consumen con una función *expect*. Es la versión en código de la tabla M: cada celda se convierte en una rama de un if.

Tabla 12. Correspondencia entre no terminales de G2 y funciones del parser

No terminal	Función	Decisión (token de anticipación)
`<programa>`	`programa()`	expect TARGET, NODE, HEX_ADDR; luego bloque()
`<bloque>`	`bloque()`	expect {, lista_instr(), expect }
`<lista_instr> / <lista_instr'>`	`lista_instr()`	ACC ∪ {IF} → otra instrucción; } → ε (fin del ciclo)
`<instruccion>`	`instruccion()`	ACC → secuencia() + expect ; ; IF → condicional()
`<secuencia> / <secuencia'>`	`secuencia()`	accion(); mientras token = -> : consumir -> y accion()
`<accion>`	`accion()`	una rama por cada palabra de ACC (R11–R18)
`<condicional> / <resto_if>`	`condicional()`	IF … THEN bloque(); si token = ELSE → bloque()
`<condicion>`	`condicion()`	ICE_LEVEL → op_rel + NUM; FIREWALL → == + estado
`<op_rel> / <estado>`	`op_rel(), estado()`	> | < | == / ACTIVE | INACTIVE

Fragmento de referencia (Python), con la documentación en inglés como en el código del proyecto:

```
class Parser:
    """Recursive-descent LL(1) parser for NetRunner (grammar G2).

    Each non-terminal is one method. The lookahead token selects the
    production using the FIRST/FOLLOW sets; terminals are consumed by expect().
    """

    ACTIONS = {"SCAN", "CONNECT", "BYPASS", "DISABLE",
               "INJECT", "EXECUTE", "EXFIL", "CLEAR"}      # ACC = FIRST(<accion>)

    def expect(self, kind):
        """Consume a terminal of the given kind or raise a syntax error."""
        tok = self.current
        if tok.kind != kind:
            raise NetRunnerSyntaxError(tok, expected=kind)
        self.advance()
        return Node(kind, lexeme=tok.lexeme)

    def lista_instr(self):
        """R3-R5: one or more instructions until FOLLOW(<lista_instr>) = { '}' }."""
        node = Node("lista_instr", [self.instruccion()])
        while self.current.kind in self.ACTIONS | {"IF"}:      # R4, else R5 (ε)
            node.add(self.instruccion())
        return node
```

```
    def secuencia(self):
        """R8-R10: <accion> ( '->' <accion> )* — left recursion already removed."""
        node = Node("secuencia", [self.accion()])
        while self.current.kind == "->":                      # R9, else R10 (ε)
            node.add(self.expect("->"))
            node.add(self.accion())
        return node

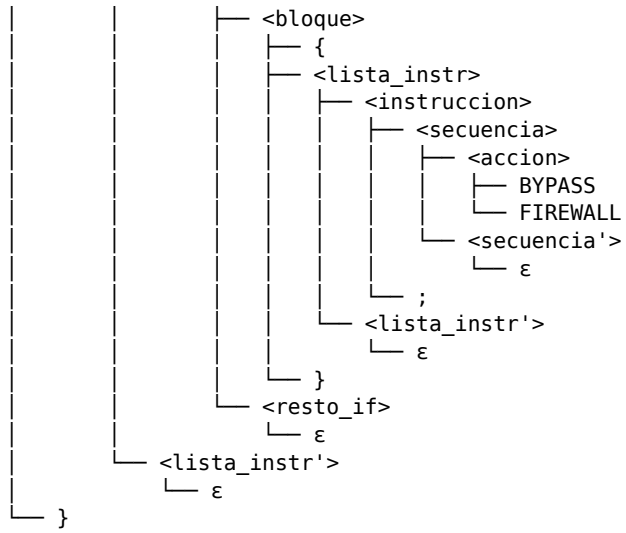
    def condicional(self):
        """R19-R21: left-factored IF ... THEN <bloque> [ELSE <bloque>]."""
        node = Node("condicional", [self.expect("IF"), self.condicion(),
                                    self.expect("THEN"), self.bloque()])
        if self.current.kind == "ELSE":                       # R20, else R21 (ε)
            node.add(self.expect("ELSE"))
            node.add(self.bloque())
        return node
```

Implementar <lista_instr′> y <secuencia′> con un *while* en lugar de una llamada recursiva es equivalente (es la forma E-BNF de la sección 2.8). Si en el árbol impreso se quieren ver explícitamente los nodos primados y las hojas ε, se usa la versión recursiva, que produce exactamente la Figura 5.

3.4 Construcción y salida del árbol de derivación

Cada función devuelve un nodo cuyo rótulo es el no terminal y cuyos hijos son, en orden, los nodos de los símbolos del lado derecho de la regla aplicada; las hojas guardan el lexema real (0xA1, 22, ...). Salida esperada para P_1 en formato de texto (versión con nodos primados):

```
<programa>
├── TARGET
├── NODE
├── 0xA1
└── <bloque>
    ├── {
    ├── <lista_instr>
    │   ├── <instruccion>
    │   │   ├── <secuencia>
    │   │   │   ├── <accion>
    │   │   │   │   ├── SCAN
    │   │   │   │   └── PORTS
    │   │   │   └── <secuencia'>
    │   │   │       ├── ->
    │   │   │       ├── <accion>
    │   │   │       │   ├── CONNECT
    │   │   │       │   ├── PORT
    │   │   │       │   └── 22
    │   │   │       └── <secuencia'>
    │   │   │           └── ε
    │   │   └── ;
    │   └── <lista_instr'>
    │       ├── <instruccion>
    │       │   └── <condicional>
    │       │       ├── IF
    │       │       ├── <condicion>
    │       │       │   ├── FIREWALL
    │       │       │   ├── ==
    │       │       │   └── <estado>
    │       │       │       └── ACTIVE
    │       │       ├── THEN
```



3.5 Manejo de errores

Errores léxicos. Ocurren cuando un AFD llega al sumidero o la entrada termina en un estado de paso. Se reportan con línea y columna y el análisis léxico continúa desde el siguiente carácter, para mostrar todos los errores de una vez. **Errores sintácticos.** Ocurren cuando la celda M[A, a] está vacía o cuando *expect* no encuentra el terminal esperado. El mensaje indica qué se esperaba y qué se encontró. Para continuar se usa **recuperación en modo pánico**: se descartan tokens hasta encontrar uno del conjunto de sincronización { ;, } } ∪ SIGUIENTE(A), y se reanuda el análisis.

Tabla 13. Catálogo de errores y su origen en el AFD o en la tabla M

Tipo	Origen formal	Ejemplo	Mensaje esperado
Léxico	Símbolo ∉ Σ (sumidero)	`SCAN@PORTS`	Símbolo '@' no pertenece al alfabeto
Léxico	Palabra fuera del trie	`scan`	'scan' no es palabra reservada
Léxico	Fin en estado de paso q8	`FIREWALL = ACTIVE`	Operador incompleto '=' (se esperaba '==')
Léxico	Fin en estado de paso q3	`0x`	Dirección hexadecimal sin dígitos
Léxico	Fin en estado de paso q5	`"rootkit`	Cadena sin comilla de cierre
Sintáctico	expect(;) en R6	`SCAN PORTS }`	Se esperaba ';' y se encontró '}'
Sintáctico	M[<accion>, ;] vacía	`SCAN PORTS -> ;`	Se esperaba una acción después de '->'
Sintáctico	M[<lista_instr>, ELSE] vacía	`{ ELSE { … } }`	ELSE sin IF: se esperaba una instrucción
Sintáctico	expect(}) en R2	`{ SCAN PORTS; (EOF)`	Se esperaba '}' y se encontró fin de archivo
Sintáctico	expect(HEX_ADDR) en R1	`TARGET NODE 255`	Se esperaba HEX_ADDR y se encontró NUM

3.6 Plan de pruebas

Los resultados esperados se verificaron con una implementación de referencia de G2 (tabla M de la Sección 2.9). Se recomienda guardar cada caso como archivo en una carpeta de pruebas y ejecutarlos todos antes de la sustentación.

Tabla 14. Casos de prueba del analizador

Caso	Descripción	Entrada	Resultado esperado
CP-01	Programa mínimo válido	`TARGET NODE 0xFF { SCAN PORTS; }`	Aceptado
CP-02	Secuencia de ataque con ->	`TARGET NODE 0x1A2B { SCAN PORTS -> CONNECT PORT 8080 -> EXFIL DATA; }`	Aceptado
CP-03	IF-THEN-ELSE con comparación	`TARGET NODE 0xAD23D { IF ICE_LEVEL > 3 THEN { BYPASS FIREWALL; } ELSE { DISABLE FIREWALL; } CLEAR LOGS; }`	Aceptado
CP-04	IF anidado y comentarios	`TARGET NODE 0x0 { ⏎ // fase de evasion ⏎ IF FIREWALL == ACTIVE THEN { IF ICE_LEVEL < 2 THEN { INJECT PAYLOAD "rootkit"; } } ⏎ EXECUTE SCRIPT "escalar.sh"; }`	Aceptado
CP-05	Falta ; al final de instrucción	`TARGET NODE 0xFF { SCAN PORTS }`	Rechazado (sintáctico)
CP-06	Secuencia termina en ->	`TARGET NODE 0xFF { SCAN PORTS -> ; }`	Rechazado (sintáctico)
CP-07	ELSE sin IF	`TARGET NODE 0xFF { ELSE { CLEAR LOGS; } }`	Rechazado (sintáctico)
CP-08	Falta } de cierre	`TARGET NODE 0xFF { SCAN PORTS;`	Rechazado (sintáctico)
CP-09	Bloque vacío	`TARGET NODE 0xFF { }`	Rechazado (sintáctico)
CP-10	Dirección sin prefijo 0x	`TARGET NODE 255 { SCAN PORTS; }`	Rechazado (sintáctico)
CP-11	Error léxico: símbolo @	`TARGET NODE 0xFF { SCAN@PORTS; }`	Rechazado (léxico)
CP-12	Error léxico: minúsculas	`TARGET NODE 0xFF { scan PORTS; }`	Rechazado (léxico)
CP-13	Un solo = en condición	`TARGET NODE 0xFF { IF FIREWALL = ACTIVE THEN { SCAN PORTS; } }`	Rechazado (léxico)

3.7 Verificación frente a la rúbrica (Arquitectura de software)

- El archivo fuente se recibe como argumento (`python main.py pruebas/cp01.nr`); no hay rutas fijas en el código.
- Las decisiones del parser salen de G2: cada función corresponde a un no terminal y cada if a una celda de M; no hay validaciones manuales que rompan el principio del parser.
- Se reportan errores léxicos y sintácticos con posición, y se continúa el análisis para listar más de uno.
- Para una entrada válida se genera e imprime el árbol de derivación sintáctica.

3.8 Trazabilidad entre fases

Tabla 15. Trazabilidad de los elementos del lenguaje

Elemento	Fase 1 (léxico)	Fase 2 (sintaxis)	Fase 3 (código)
Palabras reservadas	Trie de 111 estados	23 terminales	Conjunto/diccionario de keywords
Dirección del nodo	AFD q0→q1→q3→q4	HEX_ADDR en R1	Token HEX_ADDR, expect en programa()
Operador ->	q10 → q11	R9 (<secuencia'>)	while en secuencia()
Comparaciones	q7, q9, (q18 para <)	R24–R26 (<op_rel>)	op_rel()
Bloques { }	q12, q13	R2 (<bloque>)	bloque()
Fin de instrucción ;	q14	R6	expect(';') en instruccion()

4. Guía de sustentación e ingeniería inversa

La sustentación vale el 40 % y es individual. Esta sección resume lo que cada integrante debe poder hacer en vivo sin mirar el código.

4.1 Rastreo manual de tokens en el AFD

Tabla 16. Ejemplos de seguimiento manual en los AFD

Entrada	Recorrido	Resultado
`CONNECT`	`q0 –C→ C –O→ CO –N→ CON –N→ CONN –E→ CONNE –C→ CONNEC –T→ CONNECT`	Acepta TK_CMD
`PORTS`	`q0 –P→ P –O→ PO –R→ POR –T→ PORT (acepta) –S→ PORTS (acepta)`	Lexema más largo: PORTS
`0x1F`	`q0 –0→ q1 –x→ q3 –1→ q4 –F→ q4`	Acepta HEX_ADDR
`->`	`q0 –-→ q10 –>→ q11`	Acepta TK_OP
`=`	`q0 –=→ q8 (fin de entrada)`	Rechaza: q8 es de paso
`SCOP`	`q0 –S→ S –C→ SC –O→ sumidero`	Rechaza

4.2 Modificación de la gramática en vivo: ejemplo resuelto

Supongan que se pide agregar un ciclo `WHILE ICE_LEVEL > 2 DO { … }`. Pasos:

- **Léxico:** agregar WHILE y DO a TK_CMD (ER y trie): rama nueva W→WH→WHI→WHIL→WHILE y la transición D –O→ DO (el estado D ya existía con A e I).
- **Sintaxis:** <instruccion> ::= <secuencia> ; | <condicional> | <bucle> y <bucle> ::= WHILE <condicion> DO <bloque>.
- **Conjuntos:** PRIMERO(<instruccion>), PRIMERO(<lista_instr´>) y SIGUIENTE(<instruccion>) ganan WHILE; SIGUIENTE(<condicion>) gana DO.
- **LL(1):** WHILE no está en ACC ni es IF, así que las alternativas de <instruccion> siguen siendo disjuntas, y WHILE ∉ SIGUIENTE(<lista_instr´>) = {}. La gramática sigue siendo LL(1).
- **Código:** nueva función bucle(), una rama más en instruccion() y agregar WHILE al conjunto que usa lista_instr() para continuar el ciclo.

4.3 Preguntas probables y respuesta corta

Tabla 17. Preguntas frecuentes de sustentación

Pregunta	Respuesta
¿Por qué no basta un AFD para la sintaxis?	Las llaves anidadas balanceadas tienen la forma $a^n b^n$; un AFD tiene memoria finita y no puede contar el anidamiento. Una GIC sí, por su recursividad.
¿Qué tipo de gramática es G2 en la jerarquía de Chomsky?	Tipo 2 (independiente del contexto): todo lado izquierdo es un único no terminal.
¿Por qué la recursividad izquierda rompe el parser?	La función de A llamaría a A sin consumir tokens: recursión infinita.
¿Cómo prueban que G2 no es ambigua si la ambigüedad es indecidible?	Con una condición suficiente: G2 es LL(1) (Tablas 8 y 9, sin conflictos), y toda gramática LL(1) es no ambigua.
¿Por qué conservan reglas ε?	Eliminarlas reintroduce los prefijos comunes; el axioma no deriva ε, así que el lenguaje no cambia.
¿Derivación izquierda y derecha dan árboles distintos?	No, en una gramática no ambigua ambas corresponden al mismo árbol; solo cambia el orden de expansión.
¿Diferencia entre árbol de derivación y AST?	El de derivación muestra todas las reglas (incluye ;, { }, ε); el AST es una versión condensada sin esos detalles.
¿Qué hace el parser con un error?	Reporta línea, columna, esperado y encontrado; descarta tokens hasta ; o } y continúa (modo pánico).

5. Ajustes de consistencia recomendados para la Fase 1

Durante la revisión se encontraron detalles de la Fase 1 que conviene corregir, porque en la sustentación se puede pedir mapear errores del autómata:

- Σ_E no incluye la comilla " (necesaria para STRING) y Σ_S solo incluye el espacio, pero q17 acepta tabulaciones y saltos de línea. Al agregarlos cambia la cardinalidad total.
- La tabla de tokens incluye TK_MENOR (<), pero la Tabla 4 y su figura no tienen la transición para <; falta un estado de aceptación (por ejemplo q18).
- La sección 4.2.1 menciona ATTACK, EVADE y estados qS, qATT…, que no corresponden al trie de 23 palabras; también EVADE aparece como ejemplo de TK_CMD sin ser palabra reservada.
- La tupla del AFD lista estados qID, qGUION, qERR… que no coinciden con q0–q17 de las tablas.
- La expresión regular unificada no incluye STRING, comentarios ni espacios, aunque sí tienen AFD.
- Las figuras de operadores y de comentarios repiten los números 4 y 5, y la última tiene el título equivocado.

6. Referencias

- Tamayo Ocampo, A. M. Análisis Sintáctico: Introducción. Teoría de Lenguajes Formales, Universidad del Quindío.
- Tamayo Ocampo, A. M. Gramáticas Independientes de Contexto.

- Tamayo Ocampo, A. M. Gramáticas Bien Formadas.
- Tamayo Ocampo, A. M. BNF, E-BNF y Diagramas Sintácticos.
- Tamayo Ocampo, A. M. Árboles de derivación.
- Tamayo Ocampo, A. M. Autómatas finitos; Lenguajes regulares y expresiones regulares.
- Vargas, M. Á., Vergara, W. y López, J. P. NetRunner, Fase 1: Formalismo Léxico (2026-2).
- Aho, A., Lam, M., Sethi, R. y Ullman, J. Compiladores: principios, técnicas y herramientas (2.ª ed.). Pearson, 2008.